

SCS2314 — MIDDLEWARE ARCHITECTURE
ASSIGNMENT 4

SwiftTrack

Presentation Script for
Screencast Walkthrough

SCS2314 — Middleware Architecture Assignment 4

Group Members

Index Number	Name
23000821	R K K Jinendra
23000481	G C K S Gajanayake
23000694	J T D Jayakodi

Module: SCS2314 — Middleware Architecture
Assignment: Assignment 4 (Middleware Application)
Time Duration: Under 10 Minutes
Format: 6-Person Narrated Screencast

Script Overview

- **Total Time:** Under 10 minutes
- **Target Audience:** Lecturers and Peers
- **Language:** Simple English for clear communication

1 Speaker 1 — Introduction (1.5 min)

[Slide: Project Title and Team Names]

*"Hello everyone. We are presenting **SwiftTrack**. This is our group project for the Middleware Architecture course.*

Our team members are:

- *R K K Jinendra (23000821)*
- *G C K S Gajanayake (23000481)*
- *J T D Jayakodi (23000694)*

Speaker 4 Name

Speaker 5 Name

Speaker 6 Name

"

[Slide: The Problem]

"Swift Logistics is a growing company. They face a big problem: they use three old systems that cannot talk to each other.

- 1. The **Client System** uses an old SOAP format.*
- 2. The **Routing System** uses a modern REST format.*
- 3. The **Warehouse System** uses a special TCP message format.*

Our goal was to build a 'middle' system that connects all these different parts so they can work together smoothly."

2 Speaker 2 — Our Architecture (2 min)

[Slide: Architecture Diagram]

*"To solve this, we built a microservices system. We use **RabbitMQ** as our main messenger.*

This is how it works:

- We have a **Client Website** where users log in.
- The **API Gateway** checks if the user is allowed to enter.
- The **Orchestrator** is the brain. It saves order details in a **MongoDB** database.

We also built three **Adapters**. Think of these as 'translators'.

- One translates JSON to SOAP for the Client System.
- One talks to the Routing System using REST.
- One uses special TCP connections to talk to the Warehouse.

Finally, we have a **Notification Service**. It sends real-time updates to the website using WebSockets."

3 Speaker 3 — Why we chose this (1.5 min)

[Slide: Alternative Ideas]

"We looked at other ways to build this before choosing our final design.

First Idea: We thought about using a big central 'Bus' (ESB) like WSO2. We didn't choose this because it's a single point of failure. If the Bus breaks, everything stops. It is also very complex to maintain.

Second Idea: We thought about using 'Serverless' tools like AWS Lambda. We didn't choose this because it locks us into one company like Amazon. Also, it's not good for the Warehouse system which needs a constant connection.

Our Choice: We chose Microservices and RabbitMQ because it's open-source, easy to grow, and very reliable."

4 Speaker 4 — Handling Transactions (1.5 min)

[Slide: Saga Flow Diagram]

"A big challenge is making sure an order goes through all three systems correctly. If one part fails, we can't just leave things half-done.

We used the **Saga Pattern**. Our system follows a clear path: first CMS, then Routing, then Warehouse.

If something goes wrong at the last step, the system automatically 'undoes' the previous steps. For example, if the Warehouse is full, the system will automatically cancel the route and the inventory reservation.

This way, the data is always correct and no order is ever lost."

5 Speaker 5 — Security & Discovery (1 min)

[Slide: Security & Service List]

*"Security is very important to us. We use **JWT tokens** so only logged-in users can see their data. We also have a 'Rate Limit' to stop anyone from attacking our server with too many requests.*

*We also added a **Service Registry**. It's like a phonebook for our services. It checks every 10 seconds if each part of the system is 'healthy'. If a service stops working, the registry marks it as **DOWN** so the system knows not to use it."*

6 Speaker 6 — Live Demo (2.5 min)

[Action: Show the website in browser]

"Now, let's see the system in action. I am logging in as a client.

*You can see the **Dashboard** now. It shows order stats like 'Pending' and 'Delivered'.*

*In the top corner, it says '**Connected**'. This means we are getting live updates.*

*Let's create a **New Order**. I will add some items and a delivery address... and click Submit.*

Look! The order appears in the list. Behind the scenes, the Orchestrator talked to the CMS, the Routing system, and the Warehouse.

On the driver's side, they can see their route and mark a package as 'Delivered'. They can even take a photo or a signature as proof.

Everything works together perfectly. Thank you!"

7 Script Summary

- **Simple Words:** Replaced complex jargon like 'heterogeneous' with 'different', and 'orchestration' with 'brain' or 'coordinator' to ensure clarity.
- **Structure:** Divided into 6 parts (approx. 1-2 minutes per speaker) to involve all team members.
- **Goal:** Focuses on both the mechanical 'how' and the architectural 'why' of the middleware solution.